Self-contained Proof for Parallel Binary Tree Reduction

Sahil Bhola*

Last Edited: September 10, 2026

1 Floating-Point Model

We assume the standard floating-point model [1]: for every elementary operation $\circ \in \{+, -, \times, /\}$,

$$\mathrm{fl}(a \circ b) = (a \circ b)(1 + \delta), \qquad |\delta| \leq \mathrm{u}, \tag{1}$$

where u is the unit roundoff and δ is the relative roundoff error. Using eq. (1) for numerical analysis we typicall obtain the product

$$\prod_{i=1}^{n}(1 + \delta_i)^{\rho_i} \triangleq 1 + \theta_n, \qquad \rho = \pm 1, \tag{2}$$

where n denotes the number of accumulated roundoff error terms. For numerical analysis, we seek reliable and sharp bounds for θ_n.

2 Deterministic Rounding Error Analysis

The deterministic rounding error analysis proposes the following problem-size dependent constant

$$\gamma_n \triangleq \frac{n\mathrm{u}}{1 - n\mathrm{u}}, \tag{3}$$

for $n\mathrm{u} < 1$. Here, n is the number of accumulated error terms [1]. The problem-size dependent constant is monotonic, that is, $\gamma_{n+1} > \gamma_n$ for $\mathrm{u} > 0$. Performing backward/forward error analysis using eq. (3) we obtain

$$|\theta_n| \leq \gamma_n. \tag{4}$$

3 Parallel Binary Tree Reduction

In parallel computing, the parallel reduction kernel is routinely observed. Mathematically, the kerenel computes $y = \sum_{i=1}^{n} \mathbf{x}_i$ as

$$S_k^{\ell+1} = S_k^{\ell} + S_{k+\lceil \frac{n}{2^{\ell+1}} \rceil}^{\ell}, \qquad k = 1, \cdots, \lceil \frac{n}{2^{\ell+1}} \rceil, \tag{5}$$

such that $y = S_1^{\ell^\dagger}$. Here, $\ell^\dagger \triangleq \lceil \log_2 n \rceil$ and $S_i^0 = \mathbf{x}_i$ for all i. Note that, for some values of ℓ and n, if $S_{k+\lceil \frac{n}{2^{\ell+1}} \rceil}^{\ell}$ is unavailable, it is considered as zero. Here, we perform the rounding error analysis of the parallel binary tree reduction, re-deriving the problem-size dependent constant $\gamma_{\lceil \log_2 n \rceil}$ as proposed by [1, p. 64].

*University of Michigan, Ann Arbor

Algorithm 1 Parallel Binary-Tree Reduction

Require: Input vector $\mathbf{x} = (x_1, \ldots, x_n)$
Ensure: $y = \sum_{i=1}^{n} x_i$
1: $S_k^0 \leftarrow x_k, \quad k = 1, \ldots, n$
2: $\ell_t \leftarrow \lceil \log_2 n \rceil$
3: **for** $\ell = 0, \ldots, \ell_t - 1$ **do**
4: **for all** $k = 1, \ldots, \lceil n/2^{\ell+1} \rceil$ **in parallel do**
5: $j \leftarrow k + \lceil n/2^{\ell+1} \rceil$
6: **if** S_j^ℓ is available **then**
7: $S_k^{\ell+1} \leftarrow S_k^\ell + S_j^\ell$
8: **else**
9: $S_k^{\ell+1} \leftarrow S_k^\ell$
10: **end if**
11: **end for**
12: **end for**
13: $y \leftarrow S_1^{\ell_t}$
14: **return** y

Theorem 3.1. *Let*

$$y = \sum_{i=1}^{n} \mathbf{x}_i \tag{6}$$

be evaluated componentwise using a binary-tree reduction of depth $\ell^\dagger = \lceil \log_2 n \rceil$ *in floating-point arithmetic with unit roundoff* u. *Assume that no overflow or underflow occurs and that* $\ell^\dagger \mathrm{u} < 1$. *Then the computed result* $\hat{y}$ *satisfies*

$$\hat{y} = \sum_{i=1}^{n} (\mathbf{x}_i + \Delta \mathbf{x}_i), \qquad |\Delta \mathbf{x}_i| \leq \gamma_{\ell^\dagger} |\mathbf{x}_i|, \quad i = 1, \ldots, n, \tag{7}$$

where the inequalities are interpreted componentwise and $\gamma_{\ell^\dagger} = \ell^\dagger \mathrm{u}/(1 - \ell^\dagger \mathrm{u})$.

Proof. Using eq. (3), the computed floating-point approximation $\widehat{S}_k^{\ell+1}$ of $S_k^{\ell+1}$ is given by

$$\widehat{S}_k^{\ell+1} = \left(\widehat{S}_k^\ell + \widehat{S}_{k+\lceil \frac{n}{2^{\ell+1}} \rceil}^\ell \right) (1 + \delta_k^{\ell+1}), \qquad \left| \delta_k^{\ell+1} \right| \leq \mathrm{u}. \tag{8}$$

Since, at most we will have $\ell^\dagger$ addition operations, the computed approximation $\hat{y}$ of y is given by

$$\hat{y} = \sum_{i=1}^{n} \mathbf{x}_i \prod_{\ell=1}^{\ell^\dagger} (1 + \delta_{k_i(\ell)}^\ell), \qquad \left| \delta_{k_i(\ell)}^\ell \right| \leq \mathrm{u}, \tag{9}$$

where $k_i(\ell)$ denotes the index of the node at level ℓ corresponding to the path from the leaf containing $\mathbf{x}_i$ to the root. Now, usign eqs. (2) and (3) we obtain

$$\hat{y} = \sum_{i=1}^{n} \mathbf{x}_i + \Delta \mathbf{x}_i, \qquad \Delta \mathbf{x}_i \triangleq \theta_{\ell^\dagger} \mathbf{x}_i, \tag{10}$$

such that $|\Delta \mathbf{x}_i| \leq \gamma_{\ell^\dagger} |\mathbf{x}_i|$. □

Note, compared to the sequential dot-product, where the problem-size dependent constant is γ_n [1], we get logarithmic scaling. This shows that the binary tree reduciton algorithm has better backward stability compared to the sequential computations.

References

[1] Nicholas J Higham. *Accuracy and stability of numerical algorithms*. SIAM, 2002.